QuantumToken (QTM) Whitepaper

1. Introduction

QuantumToken (QTM) is an ERC-20 compliant token deployed on the Polygon (POL/MATIC) network. It is designed as a research-oriented asset that demonstrates how hybrid classical and post-quantum cryptographic ideas can be implemented directly inside a fungible token smart contract. Rather than focusing on aggressive tokenomics or speculation, QuantumToken prioritizes transparency, security experimentation, and verifiable on-chain behavior.

The project integrates well-known, audited building blocks from the OpenZeppelin library together with Chainlink price feeds and additional quantum-inspired verification logic. This combination creates a live environment where developers, researchers and advanced users can explore new security patterns around token transfers, minting, audits and access control.

2. Token Overview

- Name: QuantumToken
- Symbol: QTM
- Standard: ERC-20
- Network: Polygon (POL/MATIC)
- Decimals: 18

QuantumToken follows the standard ERC-20 interface, making it compatible with wallets, exchanges and DeFi protocols that support ERC-20 tokens on Polygon. On top of the base standard, the contract adds burnable and pausable extensions, as well as role-based access control.

3. Token Supply and Allocation

The total maximum supply of QuantumToken is fixed in the smart contract at:

- Max Supply: 1,000,000,000 QTM (1 billion, with 18 decimals)

The supply is allocated in two main parts:

1. Reserved Supply:

A fixed portion of 100,000,000 QTM (100 million) is minted directly to a designated reserved wallet controlled by the project owner. This allocation can be used for long-term development, strategic partnerships, research grants or other purposes disclosed by the project.

2. Public Supply:

The remaining 900,000,000 QTM (900 million) is minted to the token contract itself. These tokens are made available for public sale through an on-chain purchasing mechanism, where users can acquire QTM directly from the contract by paying in MATIC/POL.

No additional tokens can be minted beyond the hard cap of 1 billion QTM as enforced by the contract logic.

4. Primary Sale and Dynamic Pricing

QuantumToken includes a built-in primary sale function (`buyTokens`) that allows users to purchase tokens from the contract balance using MATIC/POL. The price of the token in MATIC/POL is determined using a Chainlink price feed that provides the current MATIC/USD rate.

Key characteristics of the sale mechanism include:

- A minimum price in USD per whole token (initially configured at 100 USD per token, adjusted by the number of decimals used by the price feed).

- A price increment in USD that increases the minimum price after each purchase, configured as 5 USD per whole token purchased.

- A requirement that token purchases are made in integer amounts of full tokens (multiples of 1e18 units).

When a user calls `buyTokens`:

1. The contract reads the latest MATIC/USD price from the Chainlink oracle.

2. It computes the minimum cost in USD for the requested token amount and converts this value to MATIC/POL based on the oracle price.

3. It checks that the oracle price is recent (not stale) and that the contract holds enough QTM to cover the purchase.

4. It transfers the corresponding QTM amount from the contract to the buyer.

5. It updates the minimum price in USD by adding the configured increment for each whole token sold.

6. If the buyer sent more MATIC/POL than required, the contract refunds the excess amount.

This mechanism is intended for experimentation with oracle-based pricing and is not a guarantee of secondary market prices, which are determined independently on exchanges.

5. Technology Stack and Architecture

QuantumToken is built using the following core components:

- OpenZeppelin ERC-20:

Provides a standard, audited token implementation, including transfer, approval and allowance logic.

- Burnable Extension:

Allows token holders to irreversibly destroy their own tokens, reducing the circulating supply.

- Pausable Extension:

Enables authorized roles to pause all token transfers, mints and burns in case of emergencies or critical issues.

- AccessControl:

Implements role-based permissions for administrative actions, minting, pausing, quantum verification, integration updates and audits.

- Chainlink Price Feed:

Supplies trusted, decentralized price data for MATIC/USD and also serves as a source of consensus timestamps used in the security logic.

The contract is written in Solidity version 0.8.x and uses native overflow checks provided by the compiler.

6. Roles and Governance

QuantumToken uses a set of dedicated roles to separate responsibilities within the system:

- DEFAULT_ADMIN_ROLE / QUANTUM_ADMIN:

  High-level administration, including managing roles and withdrawing accumulated MATIC/POL from the contract.

- MINTER_ROLE:

  Authorized to perform quantum-verified minting within the maximum supply limit.

- PAUSER_ROLE:

  Authorized to pause and unpause the contract in emergency scenarios.

- QUANTUM_VERIFIER_ROLE:

  Used to verify quantum hashes through classical ECDSA signatures.

- EMERGENCY_ROLE:

  Intended for handling emergency operations related to security incidents.

- WILLOW_INTEGRATOR_ROLE:

  Manages Willow integration parameters and related signatures.

- QUANTUM_AUDITOR_ROLE:

  Associated with audit and checkpoint functions inside the contract.

Roles are assigned by the admin and can be updated over time to reflect changes in governance, security practices or team structure.

7. Quantum-Inspired Verification

A distinguishing feature of QuantumToken is its quantum inspired verification subsystem, which combines classical and post-quantum style checks around certain operations such as minting and transfers.

7.1 Dual Signature Model

For selected functions (for example, `quantumMint` and `quantumTransfer`), the contract requires both:

1. Classical ECDSA Signature:

The contract verifies that a given quantum hash has been signed by an address with the QUANTUM_VERIFIER_ROLE, using the standard Ethereum signed message scheme.

2. Lamport-Style Post-Quantum Signature:

The same quantum hash is verified against a Lamport-style public key, where the root of the public key set is stored on-chain. The contract checks that the signature values match the committed public key and that each quantum hash is used only once.

By requiring both signatures, QuantumToken demonstrates how hybrid classical + post-quantum verification could be implemented in practice.

7.2 Security Scores and Levels

The contract maintains per-address security data, including:

- A security score derived from the quantum hash, existing security level and timing information.

- A security level that must reach a minimum threshold for higher alue operations.

Quantum metrics such as total operations, successful verifications, failed attempts and average complexity.

These values are updated over time as users interact with quantum-verified functions, enabling more granular control over sensitive transactions.

7.3 Audits and Checkpoints

QuantumToken periodically performs on-chain “audits” based on transaction counts and pseudo-random account selection. During these audits, the contract:

- Calculates updated security scores for selected accounts.

- Creates security checkpoints with timestamps, scores and hashes.

- May enable an enhanced security mode for accounts that consistently meet high thresholds.

This process is designed to illustrate how continuous on-chain monitoring and auditing could be implemented inside a token contract.

8. Chainlink Based Consensus Timestamp

In addition to pricing, the Chainlink price feed is used as a source of consensus timestamps. The contract reads the updatedAt value from the latest oracle round and uses it in multiple places, including:

- Enforcing timeouts for quantum verifications.
- Checking the freshness of security checks and audits.
- Throttling the frequency of operations per address.

This design shows how oracle data can be reused for both pricing and time-based security constraints.

9. Use Cases

QuantumToken is intended primarily for the following use cases:

1.Research and Education:

Serving as a live reference implementation of hybrid classical and post-quantum inspired verification techniques within an ERC-20 contract.

2. Experimental Payments and Gated Transfers:

Enabling higher-security transfer flows via `quantumTransfer`, where additional cryptographic checks and security levels are required.

3. Tooling and Analytics Demonstrations:

Providing real on-chain data (security scores, verification levels, audits, checkpoints) that can be visualized by dashboards, explorers or custom analytic tools.

While QuantumToken can be held, transferred and integrated like any other ERC-20 asset, it is not designed primarily as an investment product and does not promise returns or price performance.

10. Roadmap

The project's roadmap focuses on transparency, tooling and integrations:

Publish additional documentation and examples explaining the contract's security features, role system and Chainlink integration.

Open-source scripts and utilities to inspect quantum verifications, audit logs and security metrics on-chain.

Collaborate with developers and researchers to build demonstration dApps and dashboards around QuantumToken's data.

Pursue third-party audits of the smart contract and adjust the design based on community feedback and findings.

Any roadmap items are subject to change as the project evolves.

11. Risks and Disclaimers

QuantumToken is an experimental project and carries multiple risks, including but not limited to:

Smart contract risk:

Bugs or vulnerabilities in the code may lead to unexpected behaviour, including loss of funds.

Oracle risk:

Reliance on Chainlink price feeds means that incorrect or delayed oracle data could impact pricing and time based logic.

Market risk:

The price of QTM on secondary markets is entirely determined by supply and demand and may be highly volatile.

Regulatory risk:

Regulations related to cryptocurrencies and cryptographic systems may change over time and impact the project's operation.

Nothing in this document constitutes financial, investment, legal or tax advice. Users should perform their own research and consult qualified professionals before interacting with the token.

12. Contract Information

Contract Standard: ERC-20 (with burnable, pausable and AccessControl extensions)

Network: Polygon (POL/MATIC)

Max Supply: 1,000,000,000 QTM

Decimals: 18

Reserved Wallet: As defined in the deployed contract

Contract Version Identifier: QTM-1.0.0

The full source code of the QuantumToken smart contract is publicly available on the corresponding block explorer and/or project repository for independent review.